

FIELD GUIDE

Building AI Agents from Scratch

Everything I learned across 8 phases of building a production research agent in public.

Yash Watwani · github.com/yashwatwani/research-agent · 2026

Why I built this.

And who this is for.



Why I built this

I kept reading tutorials that showed you how to call the OpenAI API or wrap LangChain around a search tool. None of them showed you *why* each layer exists, what breaks without it, or what the tradeoffs actually are.

So I built a research agent from scratch — no framework shortcuts, no magic wrappers — and documented every decision, every bug, and every concept along the way.

Eight phases. Every commit public. Every mistake logged in LEARNINGS.md.

Who this is for

- Engineers who want to understand agents, not just use them.
- Data scientists building LLM pipelines who keep hitting the same walls.
- Anyone who asked 'why did it hallucinate?' and not had a real answer.
- People who learn by building, not by reading theory.

What this guide covers

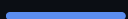
Every technical lesson from building the agent. Not a tutorial — a field guide. Concepts, tradeoffs, real bugs, and the reasoning behind every architectural decision.

The agent is open source. The code is at github.com/yashwatwani/research-agent. Read this alongside the code.

01

The agent stack.

What layers exist and why each one matters.



What an AI agent actually is

An agent is a loop. A question comes in. The agent decides what to do — search, retrieve, synthesise, repeat. It keeps going until it has an answer or gives up.

The difference between an agent and a chatbot: an agent takes actions. It calls tools, stores results, makes decisions based on what it finds. The LLM is the reasoning engine. Everything else is scaffolding.

The six layers of this agent

- **Entry:** CLI or Streamlit UI. Where the question comes in.
- **Guardrails:** Deterministic checks that wrap non-deterministic LLM behaviour.
- **Orchestration:** The async pipeline — plan, search, retrieve, synthesise.
- **Memory:** Three-tier — in-context state, vector store, knowledge graph.
- **Search:** SerpApi via an MCP server. Web as a fallback, not a default.
- **Observability:** Phoenix tracing + JSONL logs. Every LLM call visible.

Each layer is independent. You can swap pgvector for Pinecone, SerpApi for Brave, Phoenix for Datadog — the agent logic doesn't change. This is intentional.

The memory-first architecture

The most important architectural decision in this agent: check memory before searching the web.

Every web search costs a SerpApi call (limited to 100/month free). Every memory retrieval is free. The agent gets cheaper and faster the more it runs because memory fills up over time.

Memory hit path

Input filter → check memory → synthesise → output validator. Two steps skipped. No SerpApi call. No embedding call.

Memory miss path

Input filter → check memory → plan → search → retrieve → synthesise → output validator. Full pipeline. Web search only when necessary.

Why async from the start

All pipeline nodes are async — even though Phase 1 ran them sequentially. The reason: web search and OpenAI calls spend most of their time waiting for network responses. Async lets Python do other work while waiting.

Retrofitting async onto sync code is painful. Starting async is a one-time cost with compounding returns when you add parallel search queries later.

Write async from the start even if you run sequentially now. It's nearly impossible to add correctly after the fact.

AgentState — the shared state object

Every pipeline node takes an AgentState dataclass and returns one. This is the single source of truth across the entire pipeline.

```
@dataclass
class AgentState:
    question: str
    search_queries: list[str]
    search_results: list[dict]
    retrieved_context: dict
    skip_search: bool
    report: Optional[str]
    error: Optional[str]
    # Phase 7 additions
    blocked: bool
    block_reason: Optional[str]
    groundedness_score: Optional[float]
    context_text: str
```

Without a shared state object, you'd pass dozens of variables between functions. With AgentState, you pass one thing and every node can read and write everything. Adding new fields (like `blocked` in Phase 7) doesn't break any existing nodes.

State objects make pipelines readable. You can see exactly what each node needs and what it produces. This is the right pattern for any multi-step LLM pipeline.

02

Memory.

Vector store, Graph RAG, deduplication, and query routing.



Three-tier memory

The agent has three memory layers, each serving a different purpose:

Tier 1: In-context state

AgentState. Lives for one pipeline run. Contains the current question, search results, and everything the agent has found so far. Automatically garbage collected when the run ends.

Tier 2: Vector store (pgvector)

Persists across sessions. Stores text chunks as embeddings. Powers semantic search — find chunks that *mean* the same thing as the query, not just share keywords.

Tier 3: Knowledge graph (NetworkX)

Persists across sessions. Stores entities and relationships extracted from sources. Powers relationship queries — who adopted MCP? which companies use pgvector?

The memory manager (`memory_manager.py`) is the single entry point for both writing and reading. Nothing else touches the stores directly.

How vector search works

Every piece of text you store goes through this pipeline:

- Chunk the source into ~300-word pieces with 50-word overlap.
- Send each chunk to text-embedding-3-small. Get back 1536 floats.
- Store the chunk text + embedding vector in PostgreSQL via pgvector.
- At query time: embed the question, run cosine similarity against all stored chunks.
- Return the top-k chunks — the ones whose vectors point in the most similar direction.

What 1536 dimensions means

Every piece of text becomes exactly 1536 numbers regardless of length. The numbers individually mean nothing. The pattern across all 1536 encodes semantic meaning in a high-dimensional space.

Cosine similarity measures the angle between two vectors. Small angle = similar meaning, even if the words are completely different. This is how 'car' and 'automobile' return the same results.

1536 is text-embedding-3-small's output size — OpenAI chose it. It's a multiple of 512, which is computationally efficient. If you switch models, the dimension changes. That's why `EMBEDDING_DIMS` lives in `config.py`.

Graph RAG — when vector search isn't enough

Vector search returns chunks that are semantically similar to the query. It answers 'what is X?' well. It fails at 'who uses X?' and 'how are X and Y related?'

For relationship questions, you need a knowledge graph. Every source gets run through gpt-4o to extract subject→relation→object triples:

```
[
  {"subject": "Anthropic", "relation": "created", "object": "MCP"},
  {"subject": "OpenAI", "relation": "adopted", "object": "MCP"},
  {"subject": "MCP", "relation": "replaces", "object": "custom tool formats"}
]
```

These triples become edges in a NetworkX DiGraph. Query for 'who adopted MCP?' and you get every node connected to MCP, with their relation types.

The query router

Before retrieval, a classifier reads the question and returns 'graph' or 'vector'. Concept questions go to vector. Relationship questions go to graph.

Vector route

'What is X?', 'Explain Y', 'How does Z work?'
— questions about concepts and definitions.

Graph route

'Who uses X?', 'Which companies Y?', 'How are X and Y related?' — questions about relationships.

The router is what makes RAG and Graph RAG complementary rather than competing. Each handles the questions the other misses.

Deduplication — why it matters

Vector stores don't deduplicate. Insert the same chunk twice and you get two identical rows. Retrieval returns both. Your top-5 results are now top-3 unique results, polluted with repeats.

This happened in Phase 4. Same test script run twice → same chunks stored twice → retrieval results skewed. Fixed properly in Phase 7.

Two-layer dedup

The fix uses SHA-256 content hashing at two levels:

Source-level (graph store)

Hash the full source text. Check the `sources` table before running entity extraction. If seen → skip the entire gpt-4o call. This is the expensive skip — entity extraction costs the most.

Chunk-level (vector store)

Hash each chunk. Check `chunks.content_hash` before calling the embedding API. If seen → skip. Catches duplicate paragraphs across different URLs.

Why hash content, not URLs

URL-based dedup breaks two real cases: (1) same URL but the page was updated — URL check skips the update, store stays stale. (2) different URLs quoting the same Reuters paragraph — URL check stores it twice.

Dedup on what you actually stored, not where it came from. Content hashes are the principled approach.

The UNIQUE index — why it matters

A pre-check before embedding (`SELECT WHERE content_hash = ?`) handles most duplicates and saves API spend. But it's not enough alone.

If two processes run simultaneously, both can pass the `SELECT` check before either inserts. Both embed. Both try to insert. Without a database-level constraint, you get a duplicate.

```
CREATE UNIQUE INDEX chunks_content_hash_idx ON chunks (content_hash);

-- Then in the INSERT:
INSERT INTO chunks (content, content_hash, embedding, ...)
VALUES (%s, %s, %s, ...)
ON CONFLICT (content_hash) DO NOTHING;
```

The pre-check is an optimisation (avoid paying for the embedding call). The `UNIQUE` index is the actual guarantee.

Application-level checks are an optimisation. The database-level constraint is the actual guarantee. You need both.

ivfflat index — when NOT to add it

pgvector supports approximate nearest-neighbor indexing via `ivfflat`. It works by dividing all vectors into clusters and only searching within the nearest cluster.

Problem: it needs a minimum amount of data to build the clusters. With 10 rows, there are no clusters to search. Retrieval returns zero results.

Dropped the index in Phase 4. Add it back when you have 1000+ rows and full scans become slow.

Don't add performance optimisations before you have the data to justify them. Premature indexing actively breaks things.

03

Search & MCP.

SerpApi, the Model Context Protocol, and memory-first architecture.

What MCP actually is

Model Context Protocol is a standard interface for defining and calling tools. Before MCP, every agent framework had its own tool format — LangChain tools, OpenAI function calling, Anthropic tool use — all incompatible.

MCP is the USB-C of AI tools. Build a tool once, use it with any agent that speaks MCP.

In this project, MCP servers are built from scratch — not to reinvent the wheel, but to understand the protocol. Each server has two things:

```
TOOL_DEFINITION = {                # describes the tool to the LLM
    "name": "search_web",
    "description": "...",
    "parameters": { ... }
}

async def handle_tool_call(        # executes the tool
    tool_name: str,
    arguments: dict
) -> str: ...
```

The agent only sees `TOOL_DEFINITION`. It doesn't know how `handle_tool_call` works internally. This is the right abstraction — swap `SerpApi` for `Brave Search` without changing any agent logic.

Why SerpApi over the alternatives

DuckDuckGo

Blocks automated requests. HTML endpoint and JSON API both returned empty results when called from a script. Unusable.

Brave Search

Good API, but no free tier. Requires a card immediately.

SerpApi

100 free searches/month, no card needed. Returns structured results — titles, URLs, snippets — as clean JSON. The right choice for a learning project.

Memory-first — search is a fallback

The most common mistake when building agents: always searching the web. Every run hits the API, spends tokens, adds latency. The agent never gets faster.

The `check_memory_node` runs before web search. If vector similarity > 0.6 or graph returns > 2 results, skip search entirely. The agent answers from what it already knows.

The threshold is the key tuning parameter:

- Higher threshold (0.8) → agent searches more often, answers are fresher.
- Lower threshold (0.4) → agent relies on memory more, faster and cheaper.
- 0.6 is conservative — tune based on how much freshness your use case needs.

The agent gets faster and cheaper the more it runs. Memory fills up. More questions take the short path. This is the compounding return of the memory-first architecture.

04

Orchestration.

The async pipeline, conditional branching, and the agent loop.



The agent loop

The full pipeline in order:

- **input_filter_node** — validate the question. Block injections and out-of-scope.
- **check_memory_node** — retrieve from memory. Set skip_search flag.
- **plan_node** — generate 2-3 focused search queries (only if memory missed).
- **search_node** — hit SerpApi, store results with dedup (only if memory missed).
- **retrieve_node** — query router picks vector or graph path.
- **synthesise_node** — gpt-4o writes a structured report.
- **output_validator_node** — LLM judge scores groundedness.

Conditional branching

The pipeline has two paths. The skip_search flag set by check_memory_node determines which one runs:

```
async def run_agent(question: str) -> AgentState:
    state = AgentState(question=question)
    state = await input_filter_node(state)
    if state.blocked: return state          # early exit

    state = await check_memory_node(state)
    if not state.skip_search:              # only if memory missed
        state = await plan_node(state)
        state = await search_node(state)
        state = await retrieve_node(state)

    state = await synthesise_node(state)
    state = await output_validator_node(state)
    return state
```

Conditional branching is what makes the agent efficient. It only does work that's necessary. This pattern scales — add more branches as you add more capabilities.

Dual retrieval in synthesise

When the query router picks the graph path, synthesise_node doesn't just use graph results. It also pulls vector chunks as background context.

Why: graph results give you relationships (Anthropic created MCP, OpenAI adopted MCP) but no prose context. Vector chunks give you the explanatory text. The best reports combine both.

```
# in synthesise_node:
if retrieved_context['type'] == 'graph':
    # use graph triples as primary
    context_text += format_graph_results(graph_results)
    # ALSO pull vector chunks as background
    vector_results = vector_retrieve(question, top_k=3)
    context_text += format_vector_results(vector_results)
```

Why gpt-4o for planning and synthesis

plan_node uses gpt-4o to generate search queries. synthesise_node uses gpt-4o to write the report. This is deliberate — both tasks require reasoning, not just retrieval.

For classification tasks (query router, input filter classifier) we use gpt-4o-mini. Cheaper, fast enough, the task is simpler.

*Match model size to task complexity. Use the most capable model for tasks that require reasoning.
Use the cheapest model that's good enough for classification.*

Error handling in the pipeline

The agent loop wraps everything in a try/except that writes to `state.error`. This means the pipeline never crashes — it always returns an `AgentState`, even if something went wrong.

This matters for evals: if the agent crashes on one question, the eval run continues. The crashed question is recorded with `error=True` and scored as 0. The aggregate still reflects reality.

The synthesise prompt

The system prompt for synthesis is opinionated:

```
You are a research analyst. Write a clear structured report.
```

```
Format:
```

```
## Summary
```

```
2-3 sentence direct answer.
```

```
## Key findings
```

```
Bullet points of the most important facts.
```

```
## Sources
```

```
List the sources used.
```

```
Be factual. Only use information from the provided context.
```

```
Never expand abbreviations unless explicitly stated in the context.
```

The last line matters. Without it, the model expands 'MCP' to 'Multi-Cloud Platform' (wrong) or 'Master Control Program' (also wrong). Explicit constraints prevent hallucination by reducing the model's degrees of freedom.

The more specific your synthesis prompt, the less the model has to guess. Every constraint you add is a source of hallucination you've eliminated.

05

Guardrails.

Input filtering, LLM-as-judge, false positives, and dual logging.



What guardrails actually are

An LLM is non-deterministic. Same input can produce different outputs. Guardrails are deterministic checks that wrap around the non-deterministic LLM — they are the floor that makes the system predictable enough to rely on.

Three positions in the pipeline:

Before the LLM

Input filter. Validates the question before any API call. Blocks prompt injections, out-of-scope questions, and garbage input.

During tool calls

Tool allow-list. Logs if the agent tries to call a tool not in the allowed set. Soft check — log violations, don't block.

After the LLM

Output validator. Scores whether the report's claims are supported by retrieved context. Blocks if groundedness score < threshold.

LLMs alone are not production systems. Guardrails are what turn an LLM into a system you can rely on.

Two-stage input filter

Stage 1: regex patterns. Free, no API call, catches obvious cases. Runs first.

```
INJECTION_PATTERNS = [  
    r"ignore\s+(all\s+)?(previous|prior|above)\s+(instructions?|prompts?)",  
    r"you\s+are\s+now\s+(a|an)\s+",    # role hijack  
    r"reveal\s+(your|the)\s+(system\s+)?prompt",  
    ...  
]
```

Stage 2: gpt-4o-mini classifier. Runs only if regex passes. Returns allow / block / out_of_scope with a one-sentence reason.

Why two stages? Regex is free but only catches what you anticipated. The LLM catches paraphrased attacks regex misses. Running regex first means most legitimate questions never hit the LLM classifier.

Stack cheap deterministic checks first, expensive LLM checks second. Most traffic terminates at the cheap layer.

The out-of-scope verdict

The classifier has three verdicts — not two. 'out_of_scope' is different from 'block'. Out-of-scope questions get a clean explanation and 2-3 research-style suggestions. Injections get a flat block.

This distinction matters for UX. A user who asks 'what is the capital of France?' didn't do anything wrong — they just asked the wrong type of question. Tell them why and show them what the agent can do instead.

The false positive problem

A false positive is when a guardrail blocks something it shouldn't. In this project: asking about 'security vulnerabilities in MCP servers found in 2026' got blocked as 'future events'.

Why it happened: the gpt-4o-mini classifier has a training cutoff in late 2024. It sees '2026' and assumes the question is about the future — predictions, not current events.

The fix: inject the current date into the classifier's system prompt.

```
today = datetime.utcnow().strftime("%B %d, %Y")
system = f"""You are a security classifier. Today's date is {today}.
...
Questions about events in {datetime.utcnow().year} or earlier are
legitimate — do NOT block questions about current-year events."""
```

LLM classifiers are anchored to their training cutoff. Any time your application involves current-year events, inject the actual current date. The model cannot know what 'now' is otherwise.

False positives vs false negatives

False negative: guardrail lets a bad thing through. False positive: guardrail blocks a good thing. Both are failures. False positives actively harm users who are trying to do the right thing.

Logs are how you catch false positives in the wild. Every guardrail event — pass and block — is written to data/logs/guardrails.jsonl. Review it periodically.

LLM-as-judge for output validation

After the report is written, a second LLM call evaluates it. The judge sees (question, retrieved_context, report) and returns a groundedness score 0.0–1.0.

Why a separate judge call and not just a better synthesis prompt?

- The generator is biased toward producing a complete answer. It has an incentive to fill gaps.
- The judge is a fresh call with no such incentive. It just asks: is this claim in the context?
- The judge gives an explicit score you can log and tune. A prompt-only approach gives no signal.
- You can see over time whether your groundedness is improving or degrading.

Verification and generation should be separate steps. Combining them hides failure signals. You can't improve what you can't measure.

Threshold tuning

The groundedness threshold starts at 0.5. This is intentionally conservative — it's a starting point, not a final answer.

After running evals, look at the per-question groundedness scores for blocked responses. If legitimate responses are getting blocked at 0.48, lower to 0.45. If hallucinated responses are slipping through at 0.52, raise to 0.55. The logs tell you.

Soft vs hard enforcement

The tool allow-list is a soft check — it logs violations but doesn't block. The input filter and output validator are hard checks — they block on failure.

When to use soft vs hard:

Soft check (log only)

When 'wrong' is a debugging signal, not a security risk. Tool allow-list violations tell you the agent tried to call an unknown tool — useful data, not a crisis. Blast radius is low.

Hard check (block)

When 'wrong' has real consequences — injections, hallucinated reports, unauthorized actions. Block immediately. Don't log and hope.

Strictness is a function of blast radius. Match enforcement to consequences. A research agent reading public web data has different requirements than a financial agent executing trades.

Fail open vs fail closed

When the LLM classifier returns malformed JSON, the input filter logs a warning and allows the request through. This is fail-open.

The alternative — fail-closed — would block the request. For a transient LLM jitter on a research agent, that's the wrong tradeoff. Blocking a legitimate user because the classifier had a bad moment causes more harm than letting an edge case through.

Fail mode is a product decision, not just an engineering one. Think about what failure means for your specific users and use case.

Dual logging — two sinks, two jobs

Every guardrail event is logged to two places:

JSONL file (data/logs/guardrails.jsonl)

One line per event. Structured JSON. Easy to grep, filter, count, and feed into eval datasets. Answers batch questions: how many blocks today? what patterns trigger the filter?

Phoenix OTel span

The same event appears as a span in the Phoenix UI, alongside the LLM calls from that same agent run. Answers per-request questions: why was this specific request blocked?

Neither sink alone is sufficient. JSONL without Phoenix means you can't see violations in context. Phoenix without JSONL means you can't do historical analysis.

Use the right sink for the right question. Logs are not a single concern.

The session log

A separate log at data/logs/YYYY-MM-DD.log captures every complete interaction — question, full report, groundedness score, retrieved context. One file per day.

Daily files beat one giant log because you can say 'show me what happened on Tuesday' without scanning everything. They beat per-session files because 10 questions in a day shouldn't produce 10 files.

06

Evals.

Measuring quality, LLM-as-judge, and when to run them.



Guardrails vs evals — the distinction

This is the most commonly confused concept in LLM engineering. They use the same scoring logic but serve completely different purposes.

Guardrails

Online. Every request. Real time. Block or pass one response. Cost: pennies per request.

Evals

Offline. Batch. When you decide. Measure quality across a fixed set. Cost: ~\$0.30 per full run.

The practical implication: you build the groundedness scoring logic once, in `output_validator.py`. The eval suite imports and reuses it. Same criterion, online vs offline application.

Build the scoring logic once. Use it twice — online for guardrails, offline for evals.

Three scorers

Each scorer sends (question, context, report) to gpt-4o and gets back a score 0.0–1.0 with a one-sentence reasoning.

- **Relevance:** Does the report answer the actual question asked? Sees only question + report.
- **Groundedness:** Are claims supported by retrieved context? Sees question + context + report.
- **Completeness:** Does the report cover all key aspects? Sees only question + report.

Why LLM-as-judge and not reference answers

Reference answers are the obvious approach: write the ideal answer for each test question, score the agent by how close it gets. Three reasons this is wrong for research agents:

- Reference answers encode your bias frozen in time. Your ideal answer from six months ago may be wrong today.
- Research questions have 50 valid answers depending on depth, framing, and audience. One 'ideal' is arbitrary.
- Maintenance overhead. The world changes. Your reference answers don't, unless you update them.

LLM-as-judge solves all three. The judge reads the question and the context the agent actually retrieved. It asks: is this claim in the context? Does this answer the question? No ideal answer needed.

LLM-as-judge scales to any question without maintenance overhead. The judge is not checking against a gold standard — it's checking internal consistency.

Reading scores correctly

LLM judges have variance. Run the exact same eval twice and you might get 0.78 then 0.81. Some variation is noise.

- Difference under 0.05 → probably noise. Don't act on it.
- Difference of 0.05–0.10 → suggestive. Read the per-question reasoning.
- Difference over 0.10 → real change. Investigate or celebrate.

Always look per-category, not just at the aggregate. If overall score went from 0.75 to 0.78 but 'relationship' category dropped from 0.65 to 0.45, you broke Graph RAG. The aggregate hid it.

When to run evals

Not every commit. Not every day. Evals are expensive (~\$0.30 per run) and LLM variance makes small differences meaningless. Run them when something changes.

- Changed the synthesis prompt.
- Switched models (gpt-4o → gpt-4o-mini).
- Changed chunk size or overlap.
- Added a new tool or data source.
- Something feels off with the agent's outputs.
- Monthly regression check.
- Daily use → trust the guardrails.
- Every commit → too noisy, too expensive.

The eval dataset

15 questions across 6 categories: concept (generic AI/ML), domain (MCP, pgvector, Phoenix), comparison (X vs Y), relationship (exercises graph route), current (tests web search), edge (tests guardrails).

The category breakdown matters. The aggregate score hides per-category failures. Running eval-edge (2 questions) before a full run catches obvious regressions cheaply.

A smoke test before a full run is good engineering hygiene. Two questions, 30 seconds, five cents. It catches the obvious regressions before you spend 10 minutes and 30 cents on the full run.

07

Observability.

Phoenix tracing, OpenTelemetry, and seeing inside the agent.

Why observability matters for agents

An agent is a black box by default. You ask a question, you get an answer. You don't know which tools were called, how many tokens were spent, where latency came from, or why a particular output was produced.

Observability turns the black box into a glass box. Every LLM call — input, output, token count, latency — is captured and visible in the Phoenix UI.

Arize Phoenix

Self-hosted, open source LLM observability. Works by monkey-patching the OpenAI SDK — after calling `init_tracing()`, every OpenAI call is automatically intercepted and sent to Phoenix as a trace.

No code changes. No decorators. No wrappers. Just one call at startup:

```
from openinference.instrumentation.openai import OpenAIIInstrumentor
from opentelemetry.sdk.trace import TracerProvider
from opentelemetry.exporter.otlp.proto.http.trace_exporter import OTLPSpanExporter

provider = TracerProvider()
provider.add_span_processor(BatchSpanProcessor(OTLPSpanExporter(...)))
trace.set_tracer_provider(provider)
OpenAIIInstrumentor().instrument() # monkey-patches the SDK
```

Phoenix is a server, not a library. Run it in a dedicated terminal tab before anything else. Think of it like a database — it needs to be up before the application starts.

Emitting custom spans

Phoenix auto-instruments OpenAI calls. But guardrail events, dedup skips, and query routing decisions aren't LLM calls — they're pure Python. To see them in Phoenix, emit spans manually:

```
from opentelemetry import trace
_tracer = trace.get_tracer("research-agent.guardrails")

with _tracer.start_as_current_span("guardrail.input_filter") as span:
    span.set_attribute("guardrail.event_type", event_type)
    span.set_attribute("guardrail.reason", reason)
    span.set_attribute("guardrail.score", score)
```

These spans appear in Phoenix alongside the LLM calls, in the same trace. When a request fails, you see the full picture: which guardrail fired, what the LLM returned, what context was retrieved.

The port 4317 error

Phoenix needs two ports: 6006 for the UI and 4317 for the gRPC trace receiver. If you start a second script while Phoenix is already running, the second instance tries to grab port 4317 and fails.

This looks alarming but isn't. The traces still flow to the already-running Phoenix instance. The errors are the second Phoenix trying and failing to start — ignore them.

The port 4317 error means Phoenix is already running, not that tracing is broken. Check localhost: 6006 to confirm traces are arriving.

What to trace

Not everything needs a trace. Focus on what you'd actually want to see when debugging a bad output:

- Every LLM call — input, output, tokens, latency. (Auto-instrumented by Phoenix.)
- Guardrail decisions — which gate fired, what score, what reason.
- Memory routing decisions — vector vs graph, why.
- Dedup events — how many chunks skipped per source.
- Eval runs — each scorer call, each question's scores.

The test scripts (`test_agent.py`, `run_eval.py`) all call `init_tracing()` first. This means every test run produces traces in Phoenix — you can compare runs visually.

Phoenix datasets for eval comparison

After each eval run, `reporter.py` uploads the results to Phoenix as a named dataset. Two runs of the same dataset appear side-by-side in the Phoenix UI.

This is how you answer 'did the last change actually help?' without just staring at JSON files.

*Good observability is the difference between debugging by intuition and debugging by evidence.
Invest in it early. It pays back on the first hard bug.*

08

Engineering lessons.

The bugs, the decisions, and what they actually taught.

Lesson 1: Don't pre-optimize

Dropped the ivfflat index in Phase 4 because it returned zero results with small data. Added SHA-256 hashing in Phase 7 only when dedup actually became a problem. Neither was added speculatively.

Don't add performance optimisations before you have the data to justify them. Premature indexing breaks retrieval. Premature caching complicates code. Wait until the pain is real.

Lesson 2: Hash content, not identifiers

First instinct for dedup: check if we've seen this URL before. Breaks immediately for two cases: same URL with updated content (miss the update), different URLs with shared paragraphs (store duplicates).

Hash the content itself. URL is metadata. What matters is what was actually stored, not where it came from.

Design dedup around the thing you're actually deduplicating, not the label on the container.

Lesson 3: Always inject the current date

The input filter blocked 'security vulnerabilities in MCP servers found in 2026' as 'future events'. The classifier's training cutoff is 2024. It saw 2026 and concluded: prediction, not research.

Fix: one line in the system prompt. 'Today's date is {datetime.utcnow().strftime('%B %d, %Y')}.'

LLM classifiers are anchored to their training cutoff. Any task involving current events, recent developments, or the current year needs an explicit date in the prompt.

Lesson 4: python -m, always

Running 'streamlit run main.py' threw ModuleNotFoundError even though the packages were installed. The system Streamlit (from conda) was running instead of the venv one.

'python -m streamlit run main.py' always uses the active Python interpreter. No ambiguity.

Never use bare command names for tools installed inside a venv. 'python -m tool' guarantees you're using the right Python.

Lesson 5: Fail open on transient failures

When the LLM classifier returns malformed JSON, the input filter allows the request through and logs a warning. This is intentional.

A classifier returning garbled output is a transient LLM jitter, not evidence of an attack. Blocking legitimate users because the classifier had a bad moment causes more harm than the edge case.

Fail mode is a product decision. Research agent reading public data → fail open. Financial agent executing trades → fail closed. Match the failure mode to the blast radius.

Lesson 6: The DB constraint is the guarantee

The pre-check (SELECT WHERE content_hash = ?) prevents redundant embedding calls. The UNIQUE index prevents concurrent duplicates. You need both.

Two simultaneous processes can both pass the SELECT check before either inserts. Without the UNIQUE index, both insert successfully. Race condition. The pre-check alone isn't sufficient.

Application-level checks are optimisations. Database-level constraints are guarantees. Never rely on application logic alone for data integrity.

Lesson 7: record_source runs after save, not before

In `graph_store.py`, `record_source()` — which marks a source as ingested in the sources table — runs after `save_graph()` succeeds. Not before.

If entity extraction succeeds but `save_graph()` fails, the source isn't marked as ingested. A retry will work. If `record_source` ran first and `save_graph` failed, the source would be marked as done but not actually stored — silent data loss.

Mark work as complete only after it's actually complete. Optimistic logging loses data silently. Pessimistic logging wastes a retry. Pessimistic is correct.

Lesson 8: Logs answer different questions

This project has three log sinks: `guardrails.jsonl` (one line per guardrail event), `data/logs/YYYY-MM-DD.log` (one line per complete interaction), Phoenix traces (visual, per-request).

They're not redundant — they answer different questions. JSONL: how many blocks today? Phoenix: why was this specific request blocked? Session log: what did the agent say about MCP last Tuesday?

Different logs serve different questions. Don't collapse them into one. The right granularity per sink is daily for sessions, append-only for guardrails, and live trace for debugging.

Lesson 9: The suggestions pattern

When the input filter blocks a question, the agent calls gpt-4o-mini to generate 2-3 research-style alternatives on the same topic. In the Streamlit UI, these are clickable buttons that prefill the text box.

A bare 'blocked' message teaches the user nothing. Suggestions convert a dead end into a teaching moment.

Error states are UX moments. The question isn't just 'how do I stop bad input' but 'what does the user learn from this experience?'

Lesson 10: Verification and generation are separate jobs

The output validator is a second LLM call after synthesis. Why not just improve the synthesis prompt to be more grounded?

- The generator is biased toward completeness. It fills gaps.
- The validator has no such bias. It just checks.
- The validator gives a score you can log, trend, and tune.
- A prompt-only approach gives no signal — you can't tell if you're improving.

Combine generation and verification and you hide failure signals. Separate them and you can measure both independently.

09

What to build next.

The natural next steps from here.



Migrate the graph to Neo4j

NetworkX is the right starting point — zero setup, pure Python, teaches graph concepts cleanly. But it persists as a pickle file, doesn't support concurrent access, and has no query language.

When to migrate: when the graph needs to persist properly across multiple users or sessions, when you want to run complex graph queries (multi-hop traversal, path finding), or when the pickle gets large enough to slow down loads.

NetworkX is for learning graph concepts. Neo4j is for production. Build on NetworkX first, migrate when you hit its limits.

Multi-agent architecture

The current agent is a single pipeline. A natural evolution: break it into specialised agents that collaborate.

- **Planner agent** — decomposes complex questions into sub-questions.
- **Search agent** — handles web retrieval and source quality assessment.
- **Synthesis agent** — combines findings into a coherent report.
- **Critic agent** — plays devil's advocate, challenges the synthesis agent's conclusions.

Each agent has its own tools and memory. The planner coordinates. This is the architecture behind most production research systems.

Production hardening

The current setup runs on a local Mac. Moving to production requires several additions:

- **Rate limiting** — protect SerpApi quota and OpenAI spend from runaway requests.
- **Authentication** — the Streamlit UI is currently open to anyone.
- **Persistent Phoenix** — currently in-memory only, traces lost on restart. Switch to Phoenix with a PostgreSQL backend.
- **Full-content storage** — store raw source text in the sources table so you can re-chunk with different settings without re-fetching.
- **CI evals** — run make eval-edge on every PR to catch obvious regressions automatically.

Reference-based scoring for specific questions

LLM-as-judge without reference answers works well for open-ended research questions. For domain-specific questions with a single correct answer ("What year was MCP introduced?"), reference-based scoring is more reliable.

The eval suite is built to support both — add a 'reference_answer' field to dataset.py for questions where you want exact-match scoring.

No single eval approach is right for all questions. Use LLM-as-judge for open-ended, reference-based for factual. The infrastructure supports both.

BUILT IN PUBLIC

Full code. No gatekeeping.

github.com/yashwatwani/research-agent

Yash Watwani · 2026